NumPy: Practical Learning Guide

From fundamentals to the level needed for AI/ML engineering

Goal: Learn the NumPy concepts you actually need to work comfortably with numerical data, Pandas, Scikit-learn, PyTorch, and machine-learning code. You do not need to memorize the entire NumPy API.

Suggested pace: 5–7 days of focused practice, followed by using NumPy inside real exercises and ML projects.

#	Topic	Priority
1	Arrays and array creation	★★★★★
2	Array properties: shape, ndim, size, dtype	★★★★★
3	Indexing and slicing	★★★★★
4	Mathematical operations and aggregation	★★★★★
5	Axis	★★★★★
6	Reshaping and transpose	★★★★★
7	Broadcasting	★★★★★
8	Vectorization	★★★★★
9	Boolean / fancy indexing	★★★★★
10	Random numbers	★★★★■
11	Linear algebra basics	★★★★■
12	Useful functions	★★★★■
13	Advanced topics to know later	★★■■■
14	Practice exercises	★★★★★

0. Setup

Install NumPy if needed:

```
pip install numpy
```

Import it using the standard alias:

```
import numpy as np
```

Throughout this guide, **np** means the NumPy module.

1. NumPy Arrays

A NumPy array is a numerical data structure that stores values in a structured, fixed-dimensional form. The most important idea is that an array has a **shape**. A vector can be 1D, a table/matrix can be 2D, and image/tensor data can have 3 or more dimensions.

Creating arrays

```
import numpy as np

a = np.array([10, 20, 30, 40])
print(a)

matrix = np.array([
    [1, 2, 3],
    [4, 5, 6]
])
print(matrix)
```

Useful constructors

```
np.zeros((2, 3))       # 2 x 3 array of zeros
np.ones((2, 3))        # 2 x 3 array of ones
np.full((2, 3), 7)     # filled with 7
np.arange(0, 10, 2)    # [0, 2, 4, 6, 8]
np.linspace(0, 1, 5)   # 5 evenly spaced values
```

When useful: initializing matrices, test data, model inputs, numerical simulations, and ML preprocessing.

2. Array Properties

Learn to inspect an array before manipulating it. These properties are used constantly in data and ML code.

```
x = np.array([[1, 2, 3], [4, 5, 6]])

x.shape   # (2, 3) -> 2 rows, 3 columns
x.ndim    # 2 -> number of dimensions
x.size    # 6 -> total number of elements
x.dtype   # data type, e.g. int64
```

Think of **shape** as the most important piece of information. If a model expects (100, 20) and you accidentally provide (20, 100), the operation may fail or mean something completely different.

3. Indexing and Slicing

Indexing selects individual values. Slicing selects a range. For 2D arrays, the syntax is generally **array[row, column]**.

```
x = np.array([
    [10, 20, 30],
    [40, 50, 60],
    [70, 80, 90]
])

x[0, 1]      # 20
x[2, 0]      # 70
```

```
x[0, :]      # first row
x[:, 1]      # second column
x[0:2, 1:3]  # rows 0-1, columns 1-2
```

A common ML/data operation is selecting all rows but particular columns:

```
features = data[:, 0:5]
```

This means: take every row and the first five columns.

4. Mathematical Operations

NumPy performs element-wise arithmetic naturally.

```
a = np.array([1, 2, 3])
b = np.array([10, 20, 30])

a + b      # [11, 22, 33]
a - b      # [-9, -18, -27]
a * b      # [10, 40, 90]
a / b      # [0.1, 0.1, 0.1]
a ** 2     # [1, 4, 9]
```

Common aggregation functions:

```
x = np.array([1, 2, 3, 4, 5])

np.sum(x)       # 15
np.mean(x)      # 3.0
np.min(x)       # 1
np.max(x)       # 5
np.median(x)    # 3.0
np.std(x)       # standard deviation
np.argmax(x)    # index of largest value
np.argmin(x)    # index of smallest value
```

5. Axis — One of the Most Important Concepts

For a 2D array, **axis=0** operates down the rows (producing one result per column), while **axis=1** operates across columns (producing one result per row).

```
x = np.array([
    [1, 2, 3],
    [4, 5, 6]
])

np.sum(x, axis=0)
# [5, 7, 9]  -> column sums

np.sum(x, axis=1)
# [6, 15]     -> row sums

np.mean(x, axis=0)
# [2.5, 3.5, 4.5]
```

A useful mental model: **the axis you reduce disappears from the output shape.** For shape (2, 3), reducing axis 0 gives shape (3,), while reducing axis 1 gives shape (2,).

6. Reshaping

Reshaping changes how elements are arranged without changing the total number of elements. The new shape must contain the same number of elements.

```
x = np.arange(12)
# [0 1 2 3 4 5 6 7 8 9 10 11]

x.reshape(3, 4)
# shape: (3, 4)
```

```
x.reshape(2, 6)
# shape: (2, 6)

x.flatten()
# returns a flattened 1D copy

x.ravel()
# returns a flattened 1D view when possible

x.T
# transpose for a 2D array
```

Example: an image may be represented as **(height, width, channels)**. Reshaping and understanding dimensions becomes essential when preparing data for ML models.

7. Broadcasting

Broadcasting lets NumPy perform operations between arrays with compatible shapes, without manually copying values.

```
a = np.array([
    [1, 2, 3],
    [4, 5, 6]
])

b = np.array([10, 20, 30])

a + b
# [[11, 22, 33],
#  [14, 25, 36]]
```

Conceptually, NumPy applies the 1D array **b** to each row. Broadcasting follows shape-compatibility rules; learn the rules with practice rather than memorizing isolated examples.

Classic ML example: subtracting a feature-wise mean from every row:

```
X = np.array([
    [1.0, 10.0],
    [2.0, 20.0],
    [3.0, 30.0]
])

mean = X.mean(axis=0)
X_centered = X - mean
```

8. Vectorization

Vectorization means expressing an operation on whole arrays rather than writing Python loops element by element. NumPy can execute many operations efficiently in optimized low-level code.

```
x = np.array([1, 2, 3, 4, 5])

# Vectorized
y = x * 2

# Instead of conceptually doing:
# y = []
# for value in x:
#     y.append(value * 2)
```

Vectorization improves code readability and is often much faster for numerical workloads.

9. Boolean Indexing and Fancy Indexing

Boolean indexing lets you select values based on a condition.

```
x = np.array([5, 12, 3, 20, 8])
```

```
x[x > 10]
# [12 20]

x[x % 2 == 0]
# [12 20 8]

x[(x > 5) & (x < 15)]
# [12]
```

Use **&** and **|** for element-wise logical conditions, with parentheses around each condition.

Fancy indexing uses arrays/lists of indices:

```
x = np.array([10, 20, 30, 40, 50])

x[[0, 2, 4]]
# [10 30 50]
```

10. Random Numbers

Random data is useful for experiments, simulations, train/test examples, initialization, and ML demonstrations.

```
np.random.rand(2, 3)       # values in [0, 1)
np.random.randn(2, 3)      # standard-normal values
np.random.randint(1, 10, 5) # 5 random integers
np.random.choice([10, 20, 30], size=4)

# Reproducible random generator
rng = np.random.default_rng(42)
rng.integers(0, 10, size=5)
```

Using a fixed seed/generator makes experiments reproducible, which is important when debugging or comparing results.

11. Linear Algebra Basics

AI and ML rely heavily on vectors and matrices. NumPy provides the basic operations needed to understand the mathematics behind many algorithms.

Dot product

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])

np.dot(a, b)
# 1*4 + 2*5 + 3*6 = 32
```

Matrix multiplication

```
A = np.array([
    [1, 2],
    [3, 4]
])

B = np.array([
    [5, 6],
    [7, 8]
])

A @ B
# [[19, 22],
#  [43, 50]]

# np.matmul(A, B) is another way to perform matrix multiplication.
```

Other useful linear algebra functions:

```
np.linalg.norm(x)   # vector/matrix norm
np.linalg.inv(A)     # inverse, when it exists
```

```
np.linalg.solve(A, b) # solve A x = b
np.linalg.eig(A)     # eigenvalues/eigenvectors
```

For AI engineering, focus first on understanding **vectors, matrices, dot products, matrix multiplication, transpose, norms, and dimensions**. Deep mathematical mastery can come alongside ML study.

12. Useful NumPy Functions

```
np.concatenate([a, b])
np.stack([a, b])
np.vstack([a, b])
np.hstack([a, b])

np.split(x, 2)
np.unique(x)

np.sort(x)
np.argsort(x)

np.clip(x, 0, 10)

np.isnan(x)
np.isinf(x)

np.where(x > 5, 1, 0)
```

Example with where:

```
scores = np.array([35, 55, 80, 42])
result = np.where(scores >= 50, "Pass", "Fail")
# ['Fail', 'Pass', 'Pass', 'Fail']
```

13. Advanced Topics — Learn Later

These topics are useful but should not delay your fundamentals:

Topic	Why it matters
Broadcasting rules in depth	Prevents shape mistakes in numerical code
Boolean/fancy indexing details	Powerful data selection
Views vs copies	Important for memory and mutation behavior
Strides and memory layout	Useful for performance and understanding arrays
Advanced dtypes	Useful for specialized numerical/data workloads
Structured arrays	Specialized tabular/record-like data
Masked arrays	Handling masked/invalid values in specialized cases
NumPy C API	Mostly for extension/library developers

Rule: Do not spend weeks trying to master advanced NumPy before moving on. Learn advanced features when a real project requires them.

14. Mini Practice Set

Try solving these without looking at the answers first.

Exercise 1: Create a 3×4 array containing numbers 1–12.

Exercise 2: From that array, select the second row and third column.

Exercise 3: Calculate the sum of each column and each row.

Exercise 4: Reshape a 1D array of 12 elements into (4, 3).

Exercise 5: Select all values greater than 5 using boolean indexing.

Exercise 6: Add [10, 20, 30] to every row of a 2×3 matrix using broadcasting.

Exercise 7: Normalize a vector using its mean and standard deviation.

Exercise 8: Perform matrix multiplication on two compatible matrices.

Exercise 9: Find the index of the largest element using argmax.

Exercise 10: Generate a reproducible set of 10 random integers.

Example answer for Exercise 7:

```
x = np.array([10, 20, 30, 40, 50])

normalized = (x - x.mean()) / x.std()
print(normalized)
```

15. NumPy Checklist for AI/ML

Skill	Target
Create arrays	Comfortable
Understand shape/ndim/size/dtype	Comfortable
Indexing and slicing	Comfortable
Arithmetic operations	Comfortable
sum/mean/min/max/std/argmax	Comfortable
axis=0 / axis=1	Very comfortable
reshape/flatten/ravel/transpose	Comfortable
Broadcasting	Very comfortable
Vectorization	Very comfortable
Boolean indexing	Comfortable
Random generation	Comfortable
dot / @ / basic linalg	Comfortable
concatenate/stack/sort/where	Familiar
Advanced internals	Learn later

Recommended Learning Order

Python → NumPy → Pandas → Matplotlib → Scikit-learn → ML fundamentals → PyTorch → Deep Learning → Transformers/LLMs → RAG/Agents/AI Engineering

For your AI Engineering path, the practical target is not to know every NumPy function. You should be able to look at an array, understand its dimensions, manipulate its shape, select data, perform vectorized calculations, use broadcasting, and understand basic matrix operations.